

Windows Privilege Escalation: Token Impersonation

Your AlwaysInstallElevated guide closed by flagging this as a genuine shift in the series — where the first two guides exploited misconfigurations (a bad policy setting, an unquoted path), this guide and the three that follow it (SeImpersonatePrivilege, Juicy Potato, PrintSpoofer) exploit a **legitimate, deliberately-designed Windows security mechanism**, used in an unintended way. Nothing here is "broken" the way an unquoted path or a wrongly-set registry key is broken — token impersonation is a real, necessary feature of how Windows lets one process temporarily act with another security context's permissions, and the entire exploit family in this part of the series is about finding contexts where that legitimate mechanism can be triggered under attacker control. Worth locking in the distinction immediately: **Part 1 covers what access tokens actually are and why impersonation exists at all — Part 2 covers the specific tokens/privileges that make abuse possible**, since this guide is the conceptual foundation the next three guides build directly on top of, rather than a standalone exploitation technique itself.

1. What a Windows Access Token Actually Is — The Foundation

Every process and thread running on Windows carries an **access token** — a data structure that identifies the security context it's operating under: which user account it represents, which groups that user belongs to, and critically, which **privileges** (special, individually-grantable rights, conceptually similar to Linux capabilities from the companion series) that context currently holds.

Every process/thread has an associated access token containing:

- User SID (Security Identifier) - who this token represents
- Group SIDs - which groups the represented user belongs to
- Privileges - a specific list of special rights (SeDebugPrivilege, SeImpersonatePrivilege, SeBackupPrivilege, and dozens more)

When a process performs an action requiring a permission check, Windows consults the ACTING process/thread's access token to decide whether the action is allowed - the SAME core enforcement model as Linux's UID/GID/capability checks, just Windows' own distinct implementation of the underlying idea

The single fact that makes impersonation possible and necessary — Windows explicitly allows a thread to TEMPORARILY adopt a DIFFERENT access token than the one its own process was originally launched with, called impersonation, and this is a genuinely essential mechanism (a file server needs to check whether a CONNECTING CLIENT, not the file-server process's own service account, has permission to read a given file, for instance) — but the mechanism for ACQUIRING a token to impersonate, and the specific privileges controlling who's allowed to do so, is exactly where this guide's entire exploit family lives.

PART 1: Impersonation Levels & Legitimate Use Cases

2. Why Impersonation Exists — The File Server Example

```
Client connects to a file server requesting a specific file
→ the file server PROCESS itself typically runs under its OWN
  service account, NOT as the connecting client
→ but the permission check for "can THIS client read THIS
  file" needs to be evaluated using the CLIENT's identity, not
  the server process's own identity
```

```
Solution: the server THREAD handling that specific request IMPERSONATES the connecting
client's token for the duration of that operation, performs the permission-sensitive f
ile access AS the client, then reverts back to its own original context once the opera
tion completes
```

This pattern — a privileged service process temporarily adopting a LESS privileged client's identity to correctly enforce permissions on the client's behalf — is genuinely common and necessary across many Windows services (file servers, print spoolers, RPC services, and more), and its ubiquity is exactly why the exploit family covered in the rest of this series has so many viable real-world targets.

3. Impersonation Levels — How Much Access a Token Grants the Impersonator

Windows defines four distinct impersonation levels, each granting the impersonating thread progressively more capability with the borrowed token:

SecurityAnonymous → the impersonating thread can't even identify the token's owner - minimal, rarely relevant to this exploit family
SecurityIdentification → can identify/examine the token, but CANNOT actually impersonate/act as it
SecurityImpersonation → can impersonate the token FULLY on the LOCAL system - this is the level that matters most for this entire guide series, since it's what actually allows acting with the token's full local privileges
SecurityDelegation → can impersonate the token on BOTH the local system AND when making further outbound connections to OTHER remote systems - the broadest, most powerful level

The exploit family in this series (SelImpersonatePrivilege, Juicy Potato, PrintSpoofer) specifically targets scenarios where a process can obtain a token at `SecurityImpersonation` level or higher for a HIGHLY PRIVILEGED account (typically `NT AUTHORITY\SYSTEM`) — that's the entire mechanical goal every subsequent guide in this series works toward.

4. How a Token Actually Gets Obtained for Impersonation

Common legitimate paths by which a process acquires a token to impersonate (any of which can become an exploitation entry point if attacker-controllable):

- A CLIENT connects to a NAMED PIPE the attacker's process is listening on, and the client's own connection inherently carries ITS token information to the pipe server, which can then impersonate the connecting client
- DuplicateToken()/DuplicateTokenEx() Windows API calls, given an existing token handle and sufficient privilege to duplicate it
- RPC/COM-based service interactions where the CALLING process's identity gets exposed to the SERVICING process for impersonation purposes

This named-pipe connection pattern specifically is the exact mechanism your SelImpersonatePrivilege guide's Section 2 and the subsequent Juicy Potato/PrintSpoofer guides build their entire exploitation technique around — worth flagging directly here as the conceptual bridge into the rest of this series.

PART 2: The Privileges That Control Impersonation Ability

5. SelImpersonatePrivilege — The Central Privilege for This Exploit Family

Covered in full depth in its own dedicated guide immediately following this one, but worth introducing its role here structurally: `SeImpersonatePrivilege` is the specific Windows privilege that determines whether a given process/account is ALLOWED to impersonate a token at all, once one becomes available to it. Critically, this privilege is granted **by default** to several built-in service accounts that are common initial-foothold contexts in real engagements — which is precisely why this entire exploit family remains so relevant.

6. Other Impersonation-Relevant Privileges Worth Knowing

`SeAssignPrimaryTokenPrivilege` → allows assigning a token as a NEW process's PRIMARY token (rather than just impersonating it on an existing thread) - relevant to some advanced variants of this exploit family

`SeDebugPrivilege` → allows a process to open/inspect (and potentially manipulate) OTHER processes' memory and tokens directly - a significant, broader privilege with its OWN separate exploitation techniques beyond the specific impersonation-chain family this series focuses on

`SeTcbPrivilege` → "act as part of the operating system" - an extremely powerful, rarely-granted privilege that, if present, makes almost any privilege escalation technique trivial regardless of the specific method used

7. Checking Your Current Token's Privileges

```
whoami /priv
```

Example output line of direct relevance to this entire series:

Privilege Name	Description	State
SeImpersonatePrivilege	Impersonate a client after authentication	Enabled

If `SeImpersonatePrivilege` shows as Enabled (or even just Present - some contexts show it as Disabled by default but ENABLE-able), you have the specific precondition every technique in the rest of this series depends on

8. Testing Methodology

1. Run `whoami /priv` immediately upon obtaining any foothold on a Windows target, as close to a FIRST step as any enumeration action in this entire companion series

2. Specifically check for SeImpersonatePrivilege, SeAssignPrimary TokenPrivilege, SeDebugPrivilege, and SeTcbPrivilege among the results (Sections 5-6)
3. Note WHICH account/service context you're operating as - several common Windows service accounts (covered explicitly in your SeImpersonatePrivilege guide's Section 3) hold SeImpersonatePrivilege by DEFAULT, making this check disproportionately likely to succeed compared to many other findings elsewhere in either series
4. If SeImpersonatePrivilege is present, proceed directly to the SeImpersonatePrivilege guide for the specific technique/tooling this precondition unlocks

9. Prevention

- Apply the principle of least privilege to SERVICE ACCOUNT privilege assignments - don't grant SeImpersonatePrivilege (or any of Section 6's other powerful privileges) to a service account whose actual function doesn't genuinely require it
- Where a service DOES require impersonation capability for legitimate reasons (Section 2's file-server pattern), ensure the service process itself is otherwise tightly scoped and hardened, since SeImpersonatePrivilege's mere presence is the SHARED precondition every technique later in this series depends on
- Keep Windows patched - several specific tools/techniques covered in the following guides have been directly mitigated by Microsoft patches targeting the SPECIFIC exploitation mechanism (though notably NOT by removing SeImpersonatePrivilege itself, since it remains genuinely necessary for many services to function) - this is a recurring, important nuance covered in more depth in the Juicy Potato and PrintSpoofer guides specifically
- Monitor for unusual named-pipe creation and connection patterns (Section 4) as a detection layer, since this is the common underlying mechanism the following guides' specific tools rely on to actually trigger a privileged token becoming available for impersonation

10. Practical Lab Example

This guide is best studied conceptually alongside a lab VM where you've already obtained a foothold as a common Windows service account (IIS APPPOOL identity, an MSSQL service account, or similar) - these are the realistic contexts where SeImpersonatePrivilege shows up by default in practice, and seeing it in whoami /priv output on a REAL service-account foothold (rather than as an administrator, where it's trivially also present but uninteresting) is the most useful way to internalize why this privilege matters specifically.

Suggested practice order:

1. Set up a lab web application running under a service account context (IIS with an application pool identity is a common, realistic choice) and obtain a low-privilege shell through it
2. Run `whoami /priv` and confirm `SeImpersonatePrivilege` is present by default in that specific service context
3. Proceed directly into the `SeImpersonatePrivilege` guide's own lab exercise from this exact foothold, rather than starting a fresh VM - this mirrors how the technique is actually encountered in practice

Kill Chain / ATT&CK / CWE / OWASP — Full Mapping

Framework	Mapping
CWE	CWE-269 (Improper Privilege Management - the umbrella CWE for the entire token-impersonation exploit family covered across this and the next three guides)
OWASP Top 10:2025	A02:2025 Security Misconfiguration (default privilege assignment to service accounts)
CVSS	Not directly scored on its own (a foundational concept, not an exploitation technique itself) - inherits the Critical severity of the specific techniques covered in the following three guides
Cyber Kill Chain	Privilege Escalation
MITRE ATT&CK	T1134 (Access Token Manipulation) — the parent technique for this entire exploit family, with T1134.001 (Token Impersonation/Theft) as the specific sub-technique directly matching this guide's scope

Quick Reference Cheat Sheet

CHECK IMMEDIATELY ON ANY NEW FOOTHOLD:

`whoami /priv`

LOOK FOR:

<code>SeImpersonatePrivilege</code>	→ the key precondition for the next three guides in this series
<code>SeAssignPrimaryTokenPrivilege</code>	→ related, sometimes co-present
<code>SeDebugPrivilege</code>	→ broader, separate exploitation path beyond this series' scope
<code>SeTcbPrivilege</code>	→ extremely powerful, rare

IMPERSONATION LEVELS (ascending capability):

Anonymous → Identification → Impersonation → Delegation
(this exploit family targets obtaining "Impersonation" level or higher for a SYSTEM token specifically)

Root lesson: nothing in this guide is a misconfiguration - token impersonation is a genuine, necessary Windows security feature. The entire exploit family that follows exists because certain common service contexts hold the PRIVILEGE to impersonate by default, and the remaining challenge is purely mechanical: getting a SYSTEM-level token to actually become available to impersonate in the first place.
